

2 – Fundamentos de Python

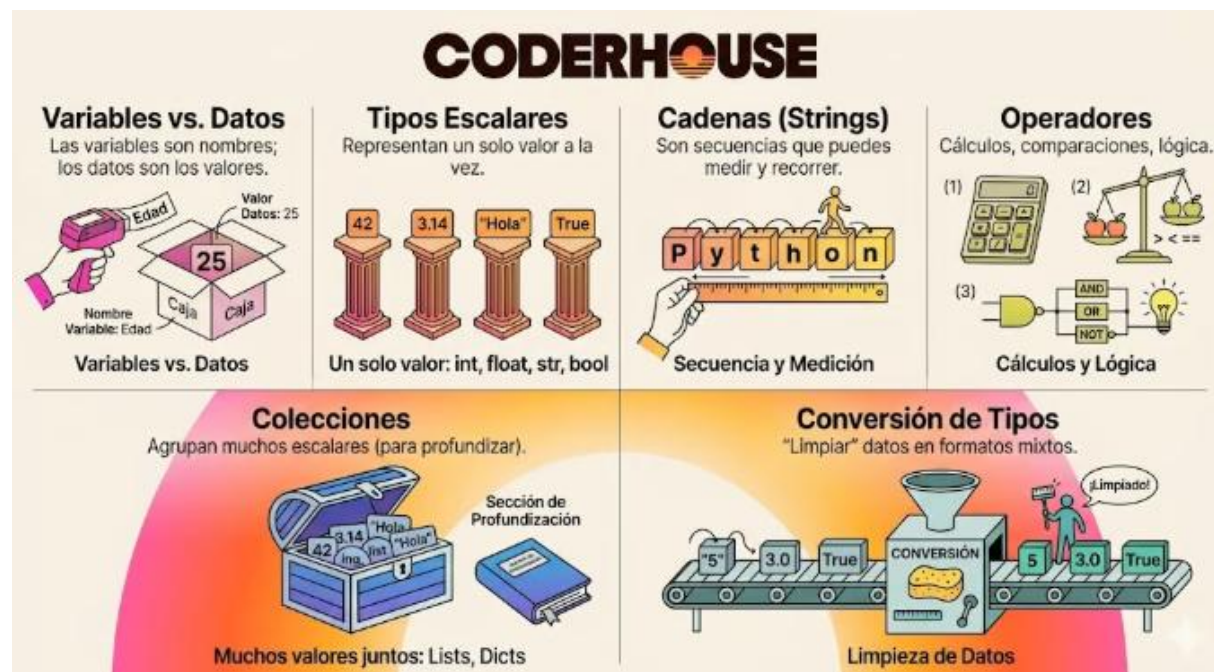
Tipos escalares, colecciones y operadores en Python, de lo simple a lo complejo

Tipos escalares, colecciones y operadores en Python

Bienvenido al inicio de tu camino técnico en la Ciencia de Datos. En módulos anteriores, exploraste el panorama de la industria y la importancia estratégica de los datos. Ahora, es momento de meter las manos en el código.

Imagina que estás construyendo un sistema para una clínica médica. Necesitas guardar el nombre de un paciente, su edad, si tiene seguro médico y los valores de sus últimos análisis de sangre. ¿Cómo sabe la computadora que un nombre es texto y una edad es un número? ¿Cómo puede comparar el valor de glucosa actual con el del mes pasado para generar una alerta? Todo esto se resuelve entendiendo los "ladrillos" fundamentales de Python: los **tipos de datos** y los **operadores**.

En esta unidad, aprenderás a distinguir entre valores simples (escalares) y agrupaciones de datos (colecciones), y cómo transformarlos usando operaciones lógicas y matemáticas.



¿Qué es una variable y qué es un dato?

Antes de operar, necesitamos contenedores. Una **variable** es, en esencia, una etiqueta que le pones a un espacio en la memoria de tu computadora para guardar un **dato**.

- **El Dato:** Es la unidad de información (ejemplo: 25, "Juan", True).
- **La Variable:** Es el nombre que usas para referirte a ese dato (ejemplo: edad, nombre_paciente, tiene_seguro).

La analogía de la etiqueta

Piensa en una variable como una **etiqueta adhesiva** y en el dato como el **objeto** al que se la pegas. Si pegas la etiqueta "Precio" a un billete de 10 dólares, el dato es el valor numérico y la etiqueta es la forma en que tu programa encontrará ese valor más tarde.

python

```
# Asignación de datos a variablesedad = 30nombre = "Lucía"
```

Es importante entender que en Python, a diferencia de otros lenguajes, no necesitas declarar qué tipo de caja vas a usar; Python mira el dato y decide automáticamente ("tipado dinámico").

Tipos escalares: Los átomos de la información

Llamamos **tipos escalares** a los valores atómicos o indivisibles. Son la unidad mínima de información con la que trabajaremos.

int (Enteros)

Representan números sin decimales. Se usan para contar unidades que no se pueden dividir.

- **Ejemplo en Data Science:** Número de transacciones, cantidad de hijos, edad en años cumplidos.
- `cantidad_productos = 500`

float (Flotantes/Decimales)

Representan números reales con precisión decimal. En Python se usa el punto `.` como separador.

- **Ejemplo en Data Science:** Precios, temperatura, promedio de ventas, probabilidad de que un cliente se dé de baja.
- `precio_dolar = 850.50`

str (Cadenas de texto / Strings)

Representan secuencias de caracteres. Siempre van entre comillas (simples `' '` o dobles `" "`).

- **Ejemplo en Data Science:** Nombres de categorías, correos electrónicos, reseñas de clientes.
- `usuario = "data_expert_2024"`

bool (Booleanos)

Solo pueden tener dos valores: `True` (Verdadero) o `False` (Falso). Son la base de la lógica computacional.

- **Ejemplo en Data Science:** ¿El cliente es premium?, ¿La columna tiene valores nulos?, ¿El modelo superó la precisión mínima?
 - `es_valido = True`
-

Escalares vs. Colecciones: La diferencia clave

Un error común para quienes inician es confundir un dato "grande" con una colección. En programación, **escalar** no significa "pequeño", significa **un solo valor**.

- **Escalar (Unidad):** Un solo valor en un momento dado. Ejemplo: `temperatura = 24.5`.
 - **Colección (Contenedor):** Una estructura que agrupa múltiples valores.
Ejemplo: `temperaturas_semana = [24.5, 22.0, 26.1, 23.4, 25.0, 21.0, 19.5]`.
Como viste en el módulo de nivelación, las **listas** y **diccionarios** son colecciones. La diferencia fundamental es que mientras un escalar es como una sola hoja de papel con un dato escrito, una colección es como una carpeta o un archivador que contiene muchas hojas relacionadas.
-

Cadenas como secuencias: El "Collar de perlas"

Aunque el tipo `str` (string) se percibe como un dato simple (el nombre de una persona), en Python se comporta como una **secuencia**.

Imagina un collar de perlas donde cada perla es una letra. Puedes mirar el collar completo (el string), pero también puedes contar las perlas o pedir "la tercera perla del collar".

python

```
ciudad = "Madrid"# Podemos saber cuántos caracteres tiene usando  
len()  
print(len(ciudad)) # Resultado: 6
```

Esta naturaleza de "secuencia" del texto es lo que nos permite, en Ciencia de Datos, limpiar nombres (quitar espacios), extraer códigos de área de números telefónicos o buscar palabras clave en tweets.

El lenguaje de transformación: Operadores

Los operadores son las instrucciones que le dicen a Python qué hacer con los datos.

A. Operadores Aritméticos (El motor matemático)

Son los que ya conoces de la escuela, con un par de invitados especiales:

- `+` (Suma)
- `-` (Resta)
- `*` (Multiplicación)
- `/` (División decimal): `7 / 2` da `3.5`
- `//` (División entera): `7 // 2` da `3` (ignora el resto)
- `%` (Módulo): `7 % 2` da `1` (es el resto de la división)
- `**` (Potencia): `2 ** 3` es `8`.

B. Operadores de Comparación (Las preguntas de sí/no)

Estos operadores siempre devuelven un **booleano** (`True` o `False`). Son esenciales para filtrar datos.

- `==` (Igual a): `5 == 5` es `True`. **¡Cuidado!** No confundas `=` (asignar valor) con `==` (comparar).
- `!=` (Diferente de): `5 != 3` es `True`.
- `>`, `<`, `>=`, `<=` (Mayor, menor, mayor o igual, menor o igual).

C. Operadores Lógicos (La toma de decisiones compleja)

Permiten combinar varias comparaciones:

- `and`: Devuelve `True` solo si **ambas** condiciones son ciertas. (Ej: El cliente es mayor de 18 **Y** tiene saldo en la cuenta).
 - `or`: Devuelve `True` si al menos **una** de las condiciones es cierta. (Ej: El pago se hace con tarjeta **O** con efectivo).
 - `not`: Invierte el sentido. `not True` es `False`.
-

Conversión de tipos y expresiones compuestas

Muchas veces recibirás datos en el formato incorrecto. Por ejemplo, un sensor puede enviarte la temperatura como texto `"25.4"` en lugar de número `25.4`. Si intentas sumarle un número al texto, Python lanzará un error.

Funciones de conversión (Casting)

- `int("10")` convierte el texto `"10"` en el número entero `10`.
- `float("10.5")` convierte el texto en decimal.

- `str(100)` convierte el número 100 en el texto "100".
- `bool(1)` devuelve `True`.

Precedencia de operadores (¿Qué va primero?)

Python sigue reglas matemáticas para resolver expresiones largas. El orden general es:

1. Paréntesis `()` (Siempre úsalos para mayor claridad).
2. Potencias `**`.
3. Multiplicación, División, Módulo.
4. Suma y Resta.
5. Comparaciones (`==`, `>`, etc.).
6. Operadores lógicos (`not`, `and`, `or`).

Ejemplo de error común: `10 + 5 * 2` es 20 (porque el `*` va antes que el `+`). Si querías que diera 30, debiste escribir `(10 + 5) * 2`.

Errores comunes y mejores prácticas

1. **Comparar tipos incompatibles:** Intentar ver si `"5" > 3`. Python no sabe cómo comparar un texto con un número. Primero debes convertir uno de ellos.
 2. **Confundir asignación con igualdad:** Usar `if edad = 18` en lugar de `if edad == 18`. El primer caso intenta cambiar la edad, el segundo hace una pregunta.
 3. **Cadenas sin comillas:** Escribir `nombre = Juan` hará que Python busque una variable llamada `Juan`. Si quieres el texto `Juan`, usa `nombre = "Juan"`.
 4. **Sobreescribir tipos:** Evita usar el mismo nombre de variable para tipos diferentes en el mismo script (ej. empezar con `dato = 10` y luego hacer `dato = "Hola"`). Aunque Python lo permite, vuelve el código muy difícil de leer y propenso a errores en Data Science.
-

Resumen de la unidad

- Las **variables** son nombres; los **datos** son los valores.
- Los tipos **escalares** (`int`, `float`, `str`, `bool`) representan un solo valor a la vez.
- Las **cadenas (strings)** son secuencias que puedes medir y recorrer.
- Las **colecciones** (que profundizarás más adelante) agrupan muchos escalares.
- Los **operadores** nos permiten hacer cálculos, comparar valores y construir lógica.
- La **conversión de tipos** es la herramienta esencial para "limpiar" datos que vienen en formatos mixtos.

Para practicar (No entregable)

Este ejercicio te permitirá consolidar la diferencia entre "guardar un dato" y "operar con él", practicando la conversión de tipos (limpieza básica) y la lógica de comparación, habilidades fundamentales para cualquier proceso de limpieza de datos en el mundo real.

Paso a paso

1. **Preparación del entorno:** En tu carpeta de proyecto del curso, crea un nuevo archivo llamado `clasificador_basico.py` o un notebook.
2. **Definición de Escalares:** Crea variables que representen los datos de un producto (ej. `nombre_producto` como `string`, `precio_unidad` como `float`, `cantidad_stock` como `int`, y `esta_disponible` como booleano).
3. **Integración con Colecciones (Repaso Incremental):** Crea una lista llamada `categorias` que contenga tres etiquetas de texto y un diccionario llamado `detalle_tecnico` con al menos tres pares clave-valor (ej. `'marca'`, `'modelo'`, `'peso'`).
4. **Operaciones y Transformaciones:**
 - Calcula el `valor_inventario` multiplicando precio por cantidad.

- Crea una variable booleana `alerta_reposicion` que sea verdadera si la cantidad es menor a un umbral (ej. 10) Y el producto está disponible.
 - Concatena el nombre del producto con su marca (extraída del diccionario) en una sola cadena de texto.
5. **Conversión Segura:** Imagina que recibes un dato de un sensor como texto: `lectura_sensor = "45.8"`. Convierte este dato a flotante y súmale 10.2.
6. **Clasificación de Salida:** Usa la función `type()` y `print()` para mostrar en consola el valor y el tipo de dato de cada resultado obtenido anteriormente.

Errores comunes a evitar

- **Error de concatenación:** Intentar sumar un string y un número sin convertir el número a string primero usando `str()`.
- **Confusión de operadores:** Usar un solo igual `=` para comparar valores en lugar de `==`.

Colecciones a Fondo

Colecciones a Fondo (Listas, Tuplas, Diccionarios Anidados y Sets para Datos)

En la unidad anterior vimos datos sueltos (un precio, un nombre). Pero en la vida real, los datos vienen en manada. Dominar las colecciones nativas de Python es lo que te permite manipular estructuras complejas (como las respuestas de una API o los registros de una base de datos) antes de pasárselos a un modelo.

Listas vs. Tuplas: El dilema de la Mutabilidad

La diferencia técnica más importante en Python es si una colección puede cambiar o no.

- **Listas [] (Mutables):** Pueden crecer, achicarse y modificar sus elementos. Son ideales para almacenar flujos de datos dinámicos (ej. una lista de ventas del día que se va actualizando).
- **Tuplas () (Inmutables):** Una vez creadas, no se pueden modificar. En Data Science las usamos para **proteger la integridad del dato**. Si una función procesa una fila de una base de datos (ej. `(id_usuario, fecha, dni)`), estructurarla como tupla garantiza que ningún bug en el código altere el DNI por accidente.

```
# Listas: Dinámicas
ventas_del_dia = [1500, 2300, 4200]
ventas_del_dia.append(3100) # Permitido

# Tuplas: Protegidas
registro_cliente = (1024, "2026-05-29", "Juan Pérez")
# registro_cliente[1] = "2026-06-01" # ✗ Lanza TypeError (No se puede alterar)
```

Diccionarios Anidados: El modelo mental de un JSON

Cuando consumes datos de una API web (como la de Spotify, Twitter o el clima), la información no vendrá en una tabla prolija; vendrá como un JSON, que en Python se traduce en un **diccionario anidado** (diccionarios dentro de diccionarios o listas).

Para extraer datos aquí, el secreto es avanzar "capa por capa" usando sus llaves (keys) o índices.

```
# Simulación de la respuesta de una API de e-commerce
api_response = {
    "status": "success",
    "data": {
        "cliente_id": 5582,
        "historial_compras": [
            {"producto": "Teclado Mecánico", "precio": 85},
            {"producto": "Mouse Pad XL", "precio": 30}
        ],
        "localizacion": {"pais": "Argentina", "provincia": "Buenos Aires"}
    }
}

# ¿Cómo extraigo el precio de la primera compra?
primer_precio = api_response["data"]["historial_compras"][0]["precio"]
print(f"El precio es: ${primer_precio}") # Salida: $85
```

Conjuntos (Sets): El arma secreta para la Limpieza de Datos

Un set {} es una colección desordenada de elementos **únicos**. En Data Science, los sets no se usan para ordenar datos, sino para dos tareas críticas:

1. **Deduplicación instantánea:** Eliminar registros duplicados de una lista en una sola línea de código.
2. **Operaciones de Conjuntos:** Cruzar listas de usuarios (Saber quiénes compraron en la campaña A Y en la campaña B).

```
# 1. Eliminar duplicados
emails_registrados = ["juan@gmail.com",
"ana@gmail.com", "juan@gmail.com", "pedro@gmail.com"]
emails_unicos = list(set(emails_registrados))
print(emails_unicos) # ['juan@gmail.com', 'ana@gmail.com', 'pedro@gmail.com']

# 2. Intersección de campañas (Clientes que están en ambas)
clientes_cybermonday = {"user_1", "user_2", "user_3"}
clientes_blackfriday = {"user_3", "user_4", "user_5"}
compraron_en_ambos = clientes_cybermonday.intersection(clientes_blackfriday)
print(compraron_en_ambos) # {'user_3'}
```

Su consigna y rúbrica están en la unidad de pre-entrega de este mismo módulo.

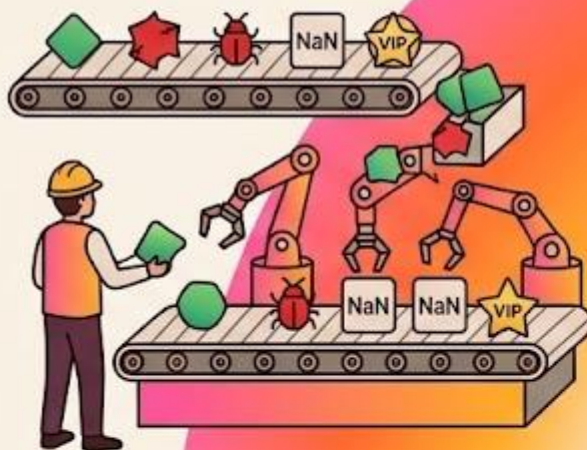
Control de Flujo: Decisiones y Bucles en Datos

Control de Flujo y Lógica Algorítmica en Data Pipelines

En el mundo real del análisis de datos, **los datos perfectos no existen**. Imagina que tu pipeline recibe miles de transacciones diarias: algunas vendrán con precios negativos por un bug del sistema, otras con campos vacíos (NaN), y otras pertenecerán a clientes VIP que requieren un cálculo de comisiones diferenciado.

Revisar esto a mano es imposible. Por eso, utilizamos el **Control de Flujo**: la capacidad de programar scripts autónomos que toman decisiones lógicas, filtran la "basura" y transforman los datos en información útil en milisegundos.

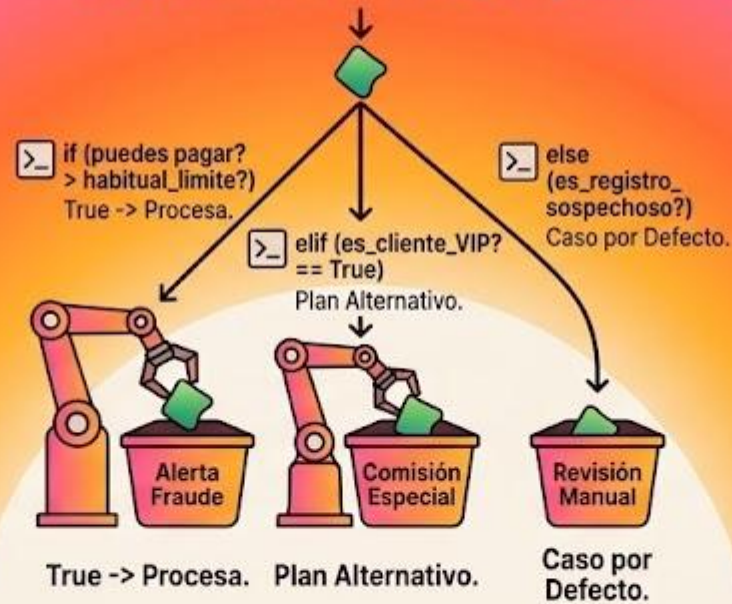
DATOS IMPERFECTOS Y CONTROL DE FLUJO



Decisiones Lógicas Autónomas

Miles de transacciones: bugs, NaNs, VIPs. El Control de Flujo filtra la 'basura' y transforma datos en milisegundos.

DECISIONES INTELIGENTES CON condicionales: if, elif, else



Segmenta clientes, captura anomalías, crea reglas complejas encadenando **elifs**.

Decisiones Inteligentes: `if`, `elif` y `else`

Un condicional es un **bifurcador de caminos lógicos**. En Data Science, lo usamos constantemente para etiquetar datos, segmentar clientes o capturar anomalías analíticas.

- **if (La condición principal):** Evalúa una expresión booleana (`True` o `False`). Si es verdadera, procesa el dato.
- **elif (Else If / El plan alternativo):** Si la primera condición fue falsa, el script evalúa esta nueva pregunta. En Data Science es vital para crear **reglas de negocio complejas** (por ejemplo, categorizar clientes por rangos de edad o niveles de gasto). Puedes encadenar tantos como tu lógica necesite.
- **else (El caso por defecto / "Catch-all"):** Si ninguna de las condiciones anteriores se cumplió, el flujo cae aquí. Es excelente para capturar errores imprevistos o registros sospechosos.

El código en la realidad (Python):

Imagina que estás automatizando la clasificación de riesgo para la aprobación de un crédito:

```
score_credificio = 650
if score_credificio >= 750:    estado = "Aprobado Automático
(Premium)"elif score_credificio >= 600:    estado = "Requiere Revisión
Manual (Riesgo Medio)"else:    estado = "Rechazado (Riesgo Alto)"
print(f"Resultado del análisis: {estado}")
```

Resultado del análisis: Requiere Revisión Manual (Riesgo Medio)

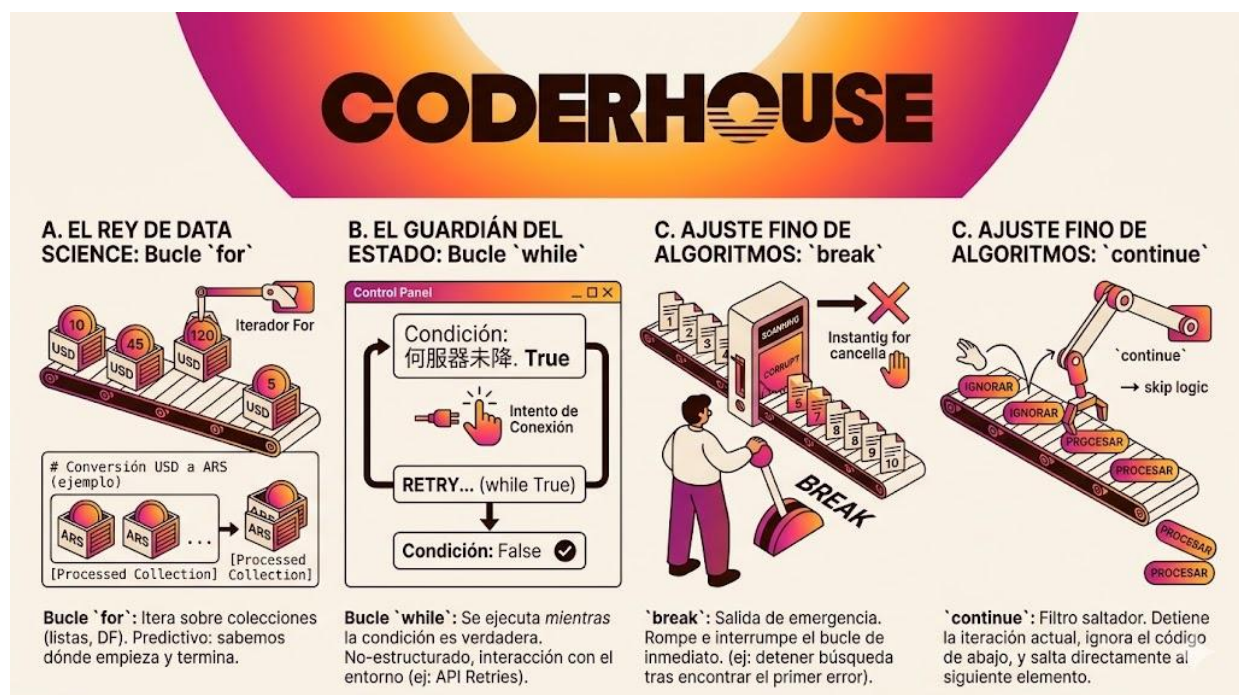
Álgebra Booleana: Lógica Combinada (and, or, not)

En la práctica, una sola condición se queda corta. Para crear filtros robustos en nuestros datasets, combinamos operadores lógicos:

- **and (Multiplicación lógica / Estricto):** Devuelve True **solo si todas** las condiciones son verdaderas.
- *Caso de uso:* ¿El usuario inició sesión **Y** la transacción supera su límite habitual? → *Disparar alerta de fraude.*
- **or (Suma lógica / Flexible):** Devuelve True si **al menos una** de las condiciones se cumple.
- *Caso de uso:* ¿El campo de email está vacío **O** no contiene un arroba @? → *Marcar registro como inválido.*
- **not (Inversión lógica):** Invierte el valor booleano. Transforma un True en False y viceversa.
- *Caso de uso:* Si el registro **NO** es nulo (not None), procesalo.

Automatización Masiva: Bucles for y while

Los bucles son el motor de la Ciencia de Datos. Nos permiten escalar el análisis de un solo dato a millones de ellos.



A. El Rey de Data Science: Bucle for

El bucle for en Python está diseñado para **iterar sobre colecciones** (listas de precios, diccionarios de usuarios, columnas de un DataFrame, archivos de texto). Su estructura es predictiva: sabemos exactamente cuándo empieza y cuándo termina (recorre el largo de la colección).

Python

```
# Pipeline de conversión de monedas (Precios en USD a ARS)
precios_usd = [10, 45, 120, 5]
tipo_cambio = 1000
precios_ars = []
for precio in precios_usd:
    conversion = precio * tipo_cambio
    precios_ars.append(conversion)
print(precios_ars)
[10000, 45000, 120000, 5000]
```

B. El Guardián del Estado: Bucle while

A diferencia del for, el bucle while se ejecuta **mientras una condición sea verdadera**. En Data Science no lo usamos tanto para recorrer datos estructurados, sino para interactuar con el entorno exterior, donde no sabemos cuántas iteraciones tomará el proceso.

- *Casos de uso clave:* * Reintentar la conexión a una API de clima si se cae el servidor (hacer un *retry* automático).
- Algoritmos de optimización matemática (como el Gradiente Descendiente) que deben repetirse hasta que el error numérico sea cercano a cero.
Ajuste Fino de Algoritmos: `break` y `continue`
No siempre queremos que un bucle corra de inicio a fin de forma ciega. Estos controladores nos dan "superpoderes" sobre la ejecución:
- **break (La salida de emergencia):** Rompe e interrumpe el bucle de inmediato, sin importar cuántos elementos queden por recorrer.
- *Uso analítico:* Estás buscando si en un archivo masivo existe al menos un registro corrupto. En cuanto el script encuentra el primero con error, ejecutas un `break`; ya no necesitas perder tiempo y cómputo procesando el resto del archivo.
- **continue (El filtro saltador):** Detiene la iteración actual, ignora el código que tiene abajo, y salta directamente al siguiente elemento del bucle.
- *Uso analítico (Data Cleaning):* Si estás procesando un dataset de clientes y encuentras una fila con datos nulos, usas `continue` para ignorar a ese usuario y pasar rápido al siguiente sin romper el script.
Ejemplo de Data Cleaning con `continue`
`registro_ventas = [150, -20, 300, 0, 450]` # El -20 es un error sistémico
`for venta in registro_ventas:` `if venta <= 0:` `continue` #
Ignora el registro negativo o en cero y pasa al siguiente # Solo
procesa las ventas válidas `impuesto = venta * 0.21` `print(f"Venta`
`válida: ${venta} | Impuesto: ${impuesto}")` `Venta válida: $150 |`
`Impuesto: $31.5``Venta válida: $300 | Impuesto: $63.0``Venta válida: $450 |`
`Impuesto: $94.5`

Funciones reutilizables para limpiar y transformar datos

Funciones reutilizables para limpiar y transformar datos

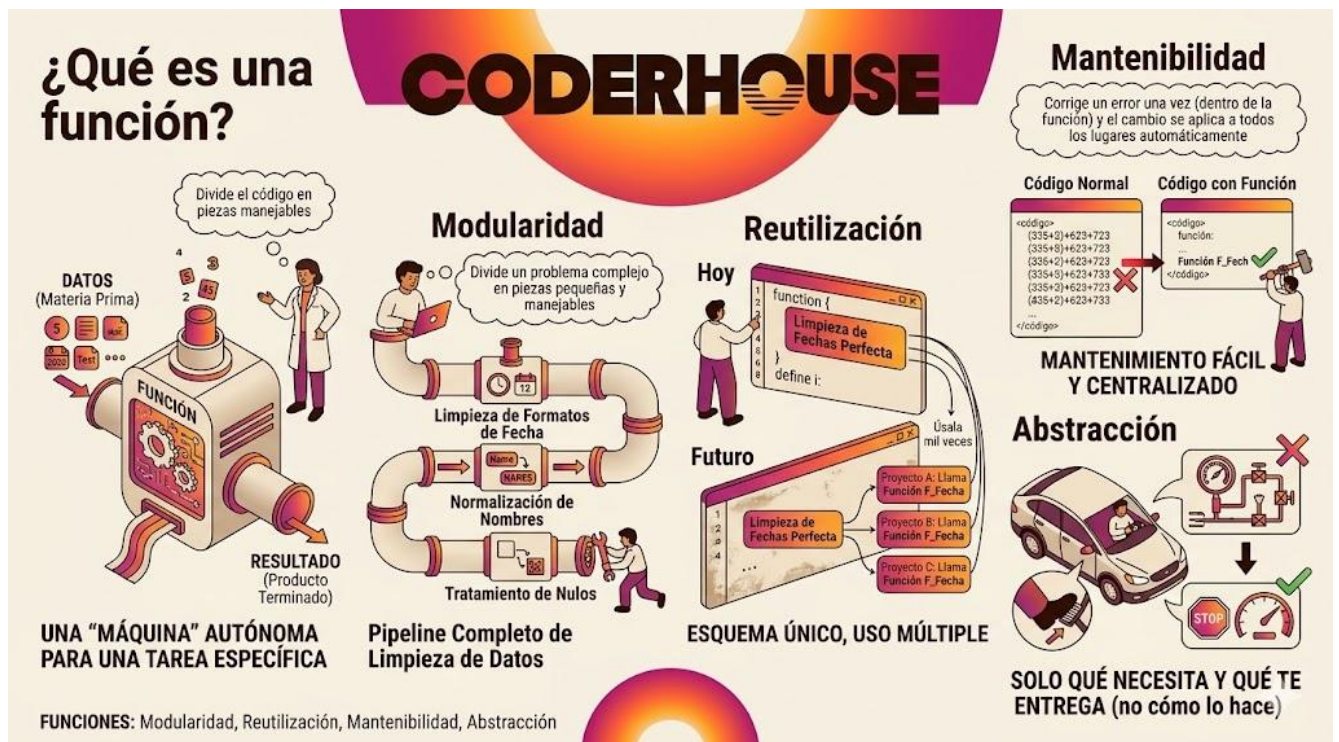
Imagina que trabajas en el equipo de análisis de una gran cadena de supermercados. Recibes archivos de ventas de 50 sucursales distintas. En todas ellas, los nombres de los productos vienen con espacios extra, algunos en mayúsculas y otros en minúsculas (ej. " manzana", "MANZANA ", "Manzana"). Además, los precios a veces traen el símbolo de moneda y otras no.

Si tuvieras que escribir el código para limpiar cada una de las 50 sucursales manualmente, terminarías con un programa gigante, difícil de leer y propenso a errores. Cada vez que el formato del archivo cambie mínimamente, tendrías que buscar y reemplazar ese cambio en 50 lugares diferentes. Aquí es donde entran las **funciones**.

Las funciones son la herramienta fundamental para aplicar el principio **DRY** (*Don't Repeat Yourself* o "No te repitas"). En esta unidad, aprenderás a encapsular tu lógica de limpieza en bloques reutilizables que harán tu trabajo como Data Scientist mucho más profesional, eficiente y escalable.

1. Qué es una función y por qué reutilizarla

En programación, una **función** es un bloque de código autónomo diseñado para realizar una tarea específica. Puedes verla como una "máquina" a la que le introduces materia prima (datos), ella realiza un proceso, y te entrega un producto terminado (el resultado).



¿Por qué son vitales en Data Science?

1. **Modularidad:** Permiten dividir un problema complejo (como un pipeline completo de limpieza de datos) en pequeñas piezas manejables.
2. **Reutilización:** Escribes la lógica una vez y la usas mil veces. Si creas una función perfecta para limpiar formatos de fecha, podrás usarla en todos tus proyectos futuros.
3. **Mantenibilidad:** Si descubres un error en tu lógica de transformación, solo tienes que corregirlo en un lugar (dentro de la función) y el cambio se aplicará automáticamente a todas las partes de tu programa que la usen.
4. **Abstracción:** Al usar una función, no necesitas recordar *cómo* hace el cálculo cada vez, solo qué necesita y qué te entrega. Es como conducir un coche: sabes que al pisar el freno se detiene, no necesitas pensar en la presión hidráulica de los frenos en ese momento.

Definición con `def` y parámetros

Para crear una función en Python utilizamos la palabra clave `def`. La estructura básica es la siguiente:

python

```
def nombre_de_la_funcion(parametro1, parametro2):    # Cuerpo de la
función (bloque indentado)    proceso = parametro1 + parametro2
return proceso
```

Componentes clave:

- **def:** Indica que estamos definiendo una función.
- **Nombre:** Debe ser descriptivo (usa verbos: `limpiar_texto`, `calcular_promedio`).
- **Parámetros:** Son los "moldes" o variables locales que recibirán los datos de entrada. Van entre paréntesis.
- **Cuerpo:** El bloque de código que realiza la tarea, siempre indentado hacia la derecha.
- **return:** La instrucción que envía el resultado de vuelta al lugar donde llamamos a la función.

Ejemplo concreto de limpieza de datos: Supongamos que queremos normalizar nombres de categorías de productos:

python

```
def normalizar_categoria(texto):    # Eliminamos espacios al
inicio/final y convertimos a minúsculas    limpio =
texto.strip().lower()    return limpio
# Uso de la función resultado = normalizar_categoria(" ELECTRONICA
") print(resultado) # Salida: "electronica"
```

Observemos que `texto` es el **parámetro** (el molde), mientras que `" ELECTRONICA "` es el **argumento** (el valor real que le pasamos).

Argumentos posicionales y con nombre

Cuando una función tiene varios parámetros, Python necesita saber qué valor corresponde a cada uno. Tenemos dos formas de hacerlo:

Argumentos Posicionales

Es la forma más común. Python asigna los valores en el mismo orden en que fueron definidos.

python

```
def crear_etiqueta(producto, precio):    return f"{producto}:
${precio}"
# "Camisa" va a producto, 25 va a precio print(crear_etiqueta("Camisa",
25))
Camisa: $25
```

Argumentos con nombre (Keyword Arguments)

Podemos especificar explícitamente a qué parámetro va cada valor. Esto hace el código mucho más legible, especialmente en Data Science donde las funciones suelen tener muchos parámetros de configuración.

python

```
# No importa el orden si usamos nombres print(crear_etiqueta(precio=40,
producto="Pantalón"))
Pantalón: $40
```

Importante: Usar argumentos con nombre ayuda a prevenir errores humanos. Imagina una función que calcula el interés de un préstamo: `calcular_interes(monto, tasa, años)`. Es mucho más seguro escribir `tasa=0.05` que simplemente pasar `0.05` y esperar que esté en la posición correcta.

`return` y la diferencia entre mostrar y devolver

Este es uno de los puntos donde más se confunden los principiantes. Es vital entender que **`print()` no es lo mismo que `return`**.

- **`print()`**: Es como hablar en voz alta. Muestra un valor en la pantalla para que el humano lo vea, pero el programa "olvida" ese valor inmediatamente. No puedes usar el resultado de un `print()` para otro cálculo.
- **`return`**: Es como escribir un resultado en una nota y entregarla. El valor regresa al programa principal, donde puede ser guardado en una variable o usado en otra operación.

Analogía: Imagina que pides una pizza.

- `print()` es que el repartidor llegue a tu puerta, te enseñe la pizza y luego se la lleve. ¡Solo pudiste verla!
- `return` es que el repartidor te entregue la caja. Ahora la pizza es tuya y puedes comerla, guardarla o compartirla.

Si una función no tiene un `return` explícito, Python devolverá por defecto el valor `None` (la representación de la nada).

Valores por defecto y múltiples retornos

Python nos ofrece flexibilidad avanzada para nuestras herramientas de datos.

Valores por defecto

A veces, un parámetro casi siempre tiene el mismo valor. Podemos predefinirlo para no tener que escribirlo siempre.

```
python
def convertir_moneda(monto, tasa=0.92): # Tasa por defecto de USD a EUR
    return monto * tasa
print(convertir_moneda(100))           # Usa 0.92 (resultado:
92.0) print(convertir_moneda(100, 0.85)) # Sobrescribe con 0.85
                                         (resultado: 85.0)
92.085.0
```

Múltiples retornos

En análisis de datos, a veces queremos que una función nos dé varios resultados a la vez (por ejemplo, el valor mínimo y el máximo de una lista).

```
python
def obtener_extremos(lista_precios):    minimo = min(lista_precios)
maximo = max(lista_precios)             return minimo, maximo # Devuelve una
tupla
p_min, p_max = obtener_extremos([10, 50, 2, 100]) print(f"El más barato
cuesta {p_min} y el más caro {p_max}")
El más barato cuesta 2 y el más caro 100
```

Ámbito local y buenas prácticas de diseño

El Ámbito (Scope)

Las variables creadas dentro de una función son **locales**. Esto significa que solo existen mientras la función se está ejecutando. Una vez que la función termina, esas variables desaparecen.

```
python
def mi_funcion():    variable_local = "Soy secreta"
print(variable_local)
mi_funcion() # print(variable_local) <-- Esto daría un ERROR porque
fuera no existe.
```

Esto es excelente porque evita que "ensuciemos" nuestro programa principal con nombres de variables temporales.

Buenas prácticas para Data Science

1. **Responsabilidad Única:** Una función debe hacer **una sola cosa** y hacerla bien. No crees una función llamada `procesar_todo()`. En su lugar, crea `limpiar_nulos()`, `convertir_tipos()` y `quitar_outliers()`.
 2. **Nombres Descriptivos:** `ajustar_precios()` es mucho mejor que `func1()`.
 3. **Docstrings:** Aunque eres principiante, acostúmbrate a poner una breve explicación de qué hace tu función justo debajo de la definición.
 4. **Evitar variables globales:** Intenta que tus funciones dependan solo de lo que entra por sus parámetros. Esto las hace predecibles y fáciles de probar.
-

Resumen de la Unidad

- Las funciones encapsulan lógica para **reutilizarla** y evitar repetir código.
- Se definen con **def**, usan **parámetros** como moldes y devuelven valores con **return**.
- **return** es fundamental para que el resultado de la función sea utilizable por el resto del script.
- Los **argumentos con nombre** y los **valores por defecto** hacen que nuestras funciones de limpieza de datos sean más robustas y fáciles de leer.
- Las variables dentro de una función tienen **ámbito local**, lo que mantiene nuestro código organizado. Dominar las funciones es pasar de escribir simples líneas de código a construir **herramientas de ingeniería de datos**. En la práctica siguiente, crearás tu propio "Kit de Primeros Auxilios" para datos usando estos conceptos.

Para practicar (No entregable)

Este ejercicio tiene como objetivo que construyas tus propias "herramientas reutilizables". En lugar de escribir código suelto, crearás un pequeño kit de funciones que podrás aplicar a cualquier lista de datos en el futuro.

Instrucciones paso a paso

1. **Preparación del entorno:**
 - Crea una carpeta en tu computadora llamada `kit_limpieza_datos`.
 - Inicia un repositorio de Git en esa carpeta y sincronízalo con un repositorio nuevo en tu cuenta de GitHub.
 - Crea un archivo llamado `limpiador.py` o un Jupyter Notebook.
2. **Implementación de las funciones:** Implementa estas 3 funciones obligatorias:
 - **Función 1: `normalizar_moneda`:** Debe recibir un string (ej: "\$1.500") y devolver un valor de tipo `float` limpio (ej: 1500.0). Usa lo aprendido sobre métodos de cadenas (`replace`) y conversión de tipos.
 - **Función 2: `validar_rango_edad`:** Debe recibir un número y dos límites (mínimo y máximo). El máximo debe tener un valor por defecto de 120. Debe devolver `True` si la edad está en el rango y `False` de lo contrario.
 - **Función 3: `limpiar_lista_texto`:** Debe recibir una lista de strings con espacios y mayúsculas desordenadas, y devolver una **nueva lista** con todos los textos en minúsculas y sin espacios extra.
3. **Pruebas (Testing):**
 - Crea una lista de prueba con datos sucios (ej: [`" ZAPATO"`, `"pantalón "`, `" MeDiAs "`]).
 - Llama a tus funciones y guarda los resultados en variables.
 - Usa `print()` para verificar que los resultados devueltos por `return` son correctos.
4. **Documentación:**
 - Escribe un pequeño comentario (`#`) arriba de cada función explicando qué parámetros recibe y qué devuelve.

Errores comunes a evitar

- **Imprimir en lugar de devolver:** Asegúrate de que tus funciones usen `return`. Si usas `print()` dentro de la función pero no `return`, al intentar guardar el resultado en una variable obtendrás `None`.
- **Nombres genéricos:** Evita nombres como `funcion1`, usa nombres que expliquen la transformación (ej. `limpiar_espacios`).

Funciones Avanzadas

Funciones Avanzadas y Reutilización de Código (Parámetros, Scope y Funciones Lambda)

A medida que tus proyectos crecen, no puedes copiar y pegar el mismo bloque de código de limpieza 20 veces. Necesitas empaquetar tu lógica en funciones profesionales, flexibles y eficientes.

Parámetros Avanzados: Flexibilidad Total

Las funciones maduras no solo reciben variables fijas; permiten configuraciones por defecto y manejan volúmenes indefinidos de argumentos mediante `*args` y `kwargs` (herramientas clave para entender cómo están programadas las librerías de Machine Learning).

FUNCIONES AVANZADAS Y REUTILIZACIÓN DE CÓDIGO (Parámetros Avanzados)

CODER



- **Argumentos por defecto:** Hacen que un parámetro sea opcional.
 - ***args y kwargs:** Permiten pasarle a la función cualquier cantidad de argumentos (posicionales o con nombre) sin tener que declararlos de antemano.
- ```
Función de limpieza flexibledef formatear_metricas(valor, moneda="USD", **detalles_extra): """Formatea métricas y agrega metadatos opcionales.""" texto_base = f"Monto: {valor} {moneda}" # Si el usuario pasa configuraciones extra, las procesamos if detalles_extra.get("notificar_gerencia") == True: texto_base += "\n ⚠ [ALERTA ALTA GERENCIA]" return texto_base
```

```
Uso básico (usa el valor por defecto)print(formatear_metricas(5000))
Monto: 5000 USD
Uso avanzado (sobreescribe la moneda y pasa un kwarg
extra)print(formatear_metricas(750000, moneda="ARS",
notificar_gerencia=True)) # Salida: Monto: 750000 ARS ⚠ [ALERTA ALTA
GERENCIA]
```

## El Peligro del Scope (Ámbito de las Variables)

Un error clásico que destruye pipelines de datos es confundir el **Ámbito Local** con el **Ámbito Global**.

- **Variable Global:** Vive fuera de las funciones, en el cuerpo principal del script. Todo el mundo la ve.
- **Variable Local:** Nace y muere *dentro* de la función. El mundo exterior no sabe que existe.

**Regla de oro en Datos:** Nunca modifiques una variable global directamente dentro de una función de limpieza. Si lo haces, pierdes la trazabilidad de cómo cambiaron tus datos (efectos secundarios).

```
BUENA PRÁCTICA: Pasar el dataset como argumento, modificar la copia
local y retornarla.dataset_global = [10, 20, 30]
def duplicar_datos(lista_datos): # lista_datos es un parámetro local
 copia_limpia = [x * 2 for x in lista_datos] return copia_limpia
nuevos_datos = duplicar_datos(dataset_global)print(dataset_global) #
[10, 20, 30] -> ¡El original está a salvo!print(nuevos_datos) # [20,
40, 60] -> El resultado esperado.
```

## Funciones Lambda: Transformaciones Express sobre la marcha

Una función **Lambda** es una función anónima, es decir, una función (sin nombre), que se escribe normalmente en una sola línea de código. No reemplaza su idea no es sustituir a las funciones normales (`def`), se usa para tareas ultra-específicas como servir como una solución breve y directa para tareas muy concretas, especialmente cuando se necesita resolver algo de "un solo uso" sin crear una estructura más larga de la necesaria.



En Python puro se, estas funciones suelen combinarse con funciones herramientas como `map()` o `filter()`. Comprender la lógica de las lambdas es el peldaño obligatorio para poder, porque encajan muy bien cuando se quiere aplicar una transformación simple o filtrar elementos de manera rápida y compacta. Comprender la lógica de las lambdas es un paso imprescindible, ya que ayuda a



interiorizar cómo funcionan este tipo de operaciones y prepara el terreno para usar el método `.apply()` cuando lleguemos a Pandas.

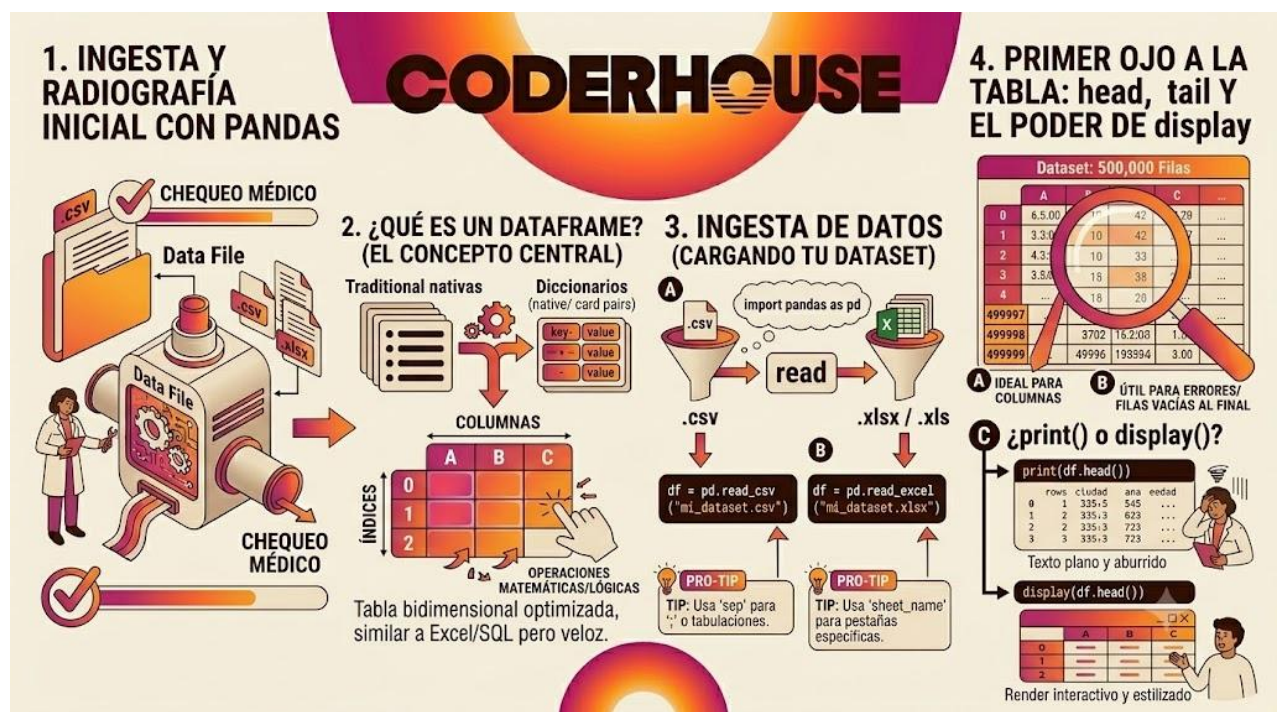
```
Sintaxis: lambda argumentos: expresion
Ejemplo 1: Una lambda simple para calcular el IVA (21%)
calcular_iva = lambda precio: precio * 1.21
print(calcular_iva(100)) # 121.0
Ejemplo 2: Aplicar una transformación express a toda una lista usando map()
precios_viejos = [100, 250, 400]
Queremos aplicarles un 10% de aumento a todos
precios_actualizados = list(map(lambda x: x * 1.10, precios_viejos))
print(precios_actualizados) # [110.0, 275.0, 440.0]
```

## Ingesta y Radiografía Inicial de Datos con Pandas

### Ingesta y Radiografía Inicial de Datos con Pandas

¡Bienvenidos a la era de la eficiencia! Hasta ahora estuvimos manipulando datos usando listas y diccionarios nativos. Pero cuando trabajamos con miles o millones de filas, necesitamos una herramienta profesional. Ahí es donde entra **Pandas**, la librería de código abierto más utilizada en el mundo para análisis y manipulación de datos.

En esta unidad aprenderás a importar el dataset que elegiste para tu pre-entrega (ya sea un archivo `.csv` o un Excel) y a realizarle un "chequeo médico" inicial utilizando comandos clave de exploración.



### El Concepto Central: ¿Qué es un DataFrame?

Antes de escribir código, necesitamos entender el modelo mental de Pandas. Pandas transforma tus archivos en una estructura llamada **DataFrame**.

Un DataFrame es, esencialmente, una tabla bidimensional (filas y columnas) con índices, muy similar a una hoja de cálculo de Excel o a una tabla SQL, pero optimizada para ejecutar operaciones matemáticas y lógicas a una velocidad increíble.

## Ingesta de Datos: Cargando tu Dataset

Para empezar a trabajar, lo primero que debemos hacer es importar la librería y leer nuestro archivo. Dependiendo del formato en el que descargaste tu dataset, usarás una función diferente:

### A. Cargar un archivo CSV (.csv)

Los archivos de valores separados por comas son los más comunes en Data Science por su ligereza.

Python

```
import pandas as pd
Cargamos el archivo CSVdf = pd.read_csv("mi_dataset.csv")
! TIP PRO: Si tu archivo usa punto y coma (;) o tabulaciones como
separador, usa el parámetro 'sep'# df = pd.read_csv("mi_dataset.csv",
sep=";")
```

### B. Cargar un archivo Excel (.xlsx o .xls)

Si tus datos provienen de una planilla de Excel tradicional, Pandas lo resuelve de forma directa.

Python

```
Nota: Puede requerir la instalación de la librería 'openpyxl' en tu
entornodf = pd.read_excel("mi_dataset.xlsx")
! TIP PRO: Si tu Excel tiene varias pestañas, puedes elegir cuál
cargar con 'sheet_name'# df = pd.read_excel("mi_dataset.xlsx",
sheet_name="Ventas2026")
```

Primer Ojo a la Tabla: head, tail y el poder de display

Si tu dataset tiene 500,000 filas, no quieres imprimirlas todas en pantalla porque colapsarías la memoria de tu navegador. Para eso usamos visualizaciones controladas.

- **df.head(n)**: Muestra las primeras n filas del DataFrame (por defecto muestra 5). Es ideal para verificar si las columnas se cargaron correctamente.
- **df.tail(n)**: Muestra las últimas n filas. Es muy útil para chequear si el archivo se cortó mal o si tiene filas vacías al final.

¿print() O display()?

En Jupyter Notebooks o Google Colab, si usas `print(df.head())`, verás el texto plano y aburrido. Si usas la función nativa `display()`, o simplemente escribes el nombre del comando al final de la celda, las plataformas e-learning renderizan el DataFrame como una **tabla interactiva y estilizada**, mucho más fácil de leer.

Python

```
Ver las primeras 5 filas de forma elegantedisplay(df.head())
Ver las últimas 3 filasdisplay(df.tail(3))
```

## La Radiografía del Dataset: Funciones de Diagnóstico

Una vez que la tabla está cargada, el científico de datos debe entender la estructura de lo que tiene enfrente. Ejecutamos estos comandos en orden:

### A. df.shape (¿Cuántos datos tengo?)

No es una función (no lleva paréntesis), es un atributo que te devuelve una tupla con la cantidad de (filas, columnas).

Python

```
print(df.shape) # Salida ejemplo: (15000, 12) -> 15 mil filas, 12
columnas.
```

### B. df.info() (El mapa de tipos de datos)

Es, probablemente, el comando más importante del diagnóstico inicial. Te dice:

1. El nombre de cada columna.
2. Cuántos datos **no nulos** hay (para detectar campos vacíos).
3. El **tipo de dato** de cada columna (int64, float64, object [texto], bool).

Python

```
df.info()
```

### C. `df.describe()` (La foto estadística)

Pandas analiza automáticamente todas las columnas numéricas de tu dataset y calcula un resumen estadístico instantáneo. En lugar de analizar las filas una por una, esta función te da una "foto panorámica" de cómo se distribuyen tus datos.

### Ejemplo de una salida en Jupyter Notebook

Imagina que cargamos un dataset de clientes de un e-commerce y ejecutamos el comando. Pandas nos devolverá una tabla exactamente como esta:

```
Ejecutamos el comandodisplay(df.describe())
```

|       | Edad    | Monto_Compra_USD |
|-------|---------|------------------|
| count | 5000.00 | 4850.00          |
| mean  | 34.20   | 120.50           |
| std   | 10.50   | 340.20           |
| min   | -5.00   | 10.00            |
| 25%   | 26.00   | 35.00            |
| 50%   | 33.00   | 65.00            |
| 75%   | 41.00   | 110.00           |
| max   | 118.00  | 15000.00         |

### ¿Qué significa cada métrica?

Vamos a desglosar fila por fila qué nos está diciendo Pandas sobre nuestro negocio:

- **count (Conteo):** Es la cantidad de registros **no nulos** que tiene esa columna.
- **Insight de datos:** Nota que Edad tiene 5000 datos, pero Monto\_Compra\_USD tiene 4850. Eso significa que hay 150 filas vacías (nulos) en la columna de montos que tendremos que limpiar después.
- **mean (Promedio / Media):** Es el valor promedio matemático de la columna (la suma de todos los valores dividida la cantidad de filas).
- **Insight de datos:** La edad promedio de nuestros compradores es de 34 años.
- **std (Desviación Estándar):** Mide qué tan dispersos o alejados están los datos respecto al promedio.
- **Insight de datos:** Una desviación baja significa que todos los datos están cerca del promedio. Una desviación alta (como en Monto\_Compra\_USD, que es de 340) nos avisa que los gastos de los usuarios son sumamente variados: tenemos gente que gasta muy poco y gente que gasta muchísimo.
- **min (Mínimo):** El valor más bajo registrado en esa columna.
- **Insight de datos: ¡Alerta de error!** El valor mínimo en edad es -5. Esto es físicamente imposible y nos indica un error de carga en el sistema que debemos corregir.
- **Los Percentiles (25%, 50%, 75%):** Imagina que ordenas a todos tus clientes en una fila, del más joven al más viejo.
- **25% (Primer Cuartil):** El corte donde se encuentra el primer cuarto de la fila. Aquí nos dice que el 25% de nuestros clientes tiene 26 años o menos.
- **50% (Mediana):** El centro exacto de los datos. El 50% de los clientes tiene menos de 33 años, y el otro 50% es mayor de 33.
- **75% (Tercer Cuartil):** El corte del 75%. El 75% de los usuarios tiene 41 años o menos.
- **max (Máximo):** El valor más alto registrado.
- **Insight de datos:** El máximo de edad es 118 (sospechoso, podría ser un dato mal cargado) y el gasto máximo es 15000 (un cliente VIP o un error).

```
Muestra la estadística descriptiva de las variables numéricasdisplay(df.describe())
```



## D. `df.isnull().sum()` (El detector de agujeros)

La función `df.isnull()` devuelve una máscara de True/False para cada celda. Si le sumamos `.sum()`, Pandas cuenta cuántos datos faltantes (NaN) hay en cada columna. Es el paso obligatorio antes de empezar la limpieza de datos.

```
Contamos cuántos datos nulos/vacíos hay por columna
print("Reporte de Datos Faltantes:")
print(df.isnull().sum())
```

Opción 1: El truco "Pro" (Usando `.mean()`)

Esta es la forma más elegante y *pythonica*. Como `df.isnull()` devuelve valores booleanos (True donde hay nulo, False donde no), y para Python el True vale 1 y el False vale 0, si calculamos el **promedio** (`.mean()`), obtenemos directamente la proporción de nulos. Solo falta multiplicarlo por 100.

```
Calcular el porcentaje de nulos por columna
porcentaje_nulos = df.isnull().mean() * 100
print(porcentaje_nulos)
```

Opción 2: La forma explícita (Regla de 3 simple)

Si sientes que a tus alumnos les va a costar entender el truco del promedio, puedes ir por la vía tradicional: dividir la suma de nulos por el total de filas (`len(df)`) y multiplicarlo por 100.

```
Suma de nulos dividido el total de filas, por 100
porcentaje_nulos = (df.isnull().sum() / len(df)) * 100
print(porcentaje_nulos)
```

### Crear una Tabla de Diagnóstico

Para poder ver tus datos de forma profesional puedes calcular porcentaje de nulos, convertirlos en un dataframe y ordenar las columnas de mayor a menor

```
1. Calculamos el porcentaje y lo ordenamos de mayor a menor
reporte_nulos = df.isnull().mean() * 100
reporte_nulos = reporte_nulos.sort_values(ascending=False)
2. Lo convertimos en un DataFrame para que se vea lindo con display()
df_nulos = pd.DataFrame(reporte_nulos, columns=['Porcentaje de Nulos (%)'])
3. Mostramos el resultado
display(df_nulos)
```

¿Cómo se vería la salida en Jupyter?

| Columna           | Porcentaje de Nulos (%) |
|-------------------|-------------------------|
| ingresos_anuales  | 42.15                   |
| fecha_nacimiento  | 12.03                   |
| historial_credito | 5.40                    |
| id_cliente        | 0.00                    |

# Segunda Pre-Entrega: Ingesta, Radiografía y Funciones de Transformación

## Actividad

### Segunda Pre-Entrega: Ingesta, Radiografía y Funciones de Transformación

#### Objetivo de la Entrega

El objetivo de esta segunda instancia es poner en práctica tus habilidades de programación en Python y exploración inicial de datos. Deberás abrir tu entorno de desarrollo (Jupyter Notebook o Google Colab), importar el dataset seleccionado en la Pre-entrega 1, realizar un diagnóstico técnico completo de su estructura y programar tus primeras funciones personalizadas para limpiar o transformar variables clave.

#### ¿Qué debes entregar?

Debes presentar un archivo de Jupyter Notebook (`.ipynb`) o el enlace a tu cuaderno de Google Colab (con acceso público de lectura). El código debe estar limpio, comentado y estructurado en los siguientes **4 bloques obligatorios**:

## 1. Ingesta de Datos y Primer Vistazo

- Importa la librería Pandas.
- Carga tu dataset (en formato `.csv` o `.xlsx`) en un DataFrame llamado `df`.
- Utiliza la función adecuada para mostrar en pantalla las primeras **5 filas** de tus datos de manera estilizada.

## 2. Radiografía Técnica del Dataset

Ejecuta los comandos de diagnóstico necesarios para responder programáticamente a las siguientes preguntas en tu notebook:

- ¿Cuáles son las dimensiones exactas del dataset (filas y columnas)?
- ¿Qué tipos de datos procesa cada columna? ¿Son todos correctos o hay variables numéricas que se leyeron como texto?
- Obtén la foto estadística (`describe`) de tus variables numéricas. Escribe una breve celda de texto (Markdown) comentando al menos **dos hallazgos o anomalías** que notes en ese resumen (por ejemplo: si notas valores mínimos imposibles, desviaciones estándar muy altas o presencia de *outliers*).

## 3. Diagnóstico de Datos Faltantes

- Desarrolla un bloque de código que calcule y muestre de forma ordenada (de mayor a menor) el **porcentaje exacto de datos nulos** que tiene cada una de las columnas de tu dataset.

## 4. Lógica Algorítmica y Funciones Personalizadas (Python Puro)

Para demostrar el dominio de los fundamentos de Python, debes crear e implementar funciones aplicadas a tu dataset:

- **Función Formal (`def`):** Diseña una función propia que reciba un parámetro, aplique una estructura de control de flujo condicional (`if/elif/else`) basada en una regla de negocio de tu proyecto, y retorne un nuevo valor.
- **Función Express (`lambda`):** Diseña una función lambda para realizar una transformación matemática o de texto rápida sobre otra columna.
- *Nota:* Aplica estas funciones a tus datos (puedes usar el método `.apply()` de Pandas o estructuras nativas) para generar columnas nuevas o transformar las existentes.

Ejemplo de cómo debe verse tu entrega (Formato Notebook):

```
=====# BLOQUE 1: INGESTA DE
DATOS# =====import pandas as pd
df = pd.read_csv("ecommerce_customer_data.csv")display(df.head())
=====# BLOQUE 2: RADIOGRAFÍA
TÉCNICA# =====print(f"Dimensiones
del dataset: {df.shape}")df.info()display(df.describe())
```

Entregable

Segunda Pre-Entrega: Ingesta, Radiografía y Funciones

El propósito de este entregable es demostrar tu capacidad para integrar librerías fundamentales de Python (Pandas) con lógica de programación pura para realizar un diagnóstico inicial de un dataset real.

Pasos a seguir:

1. **Preparación del Entorno:** Crea un nuevo cuaderno de Jupyter (.ipynb) en Google Colab o tu entorno local. Asegúrate de tener a mano el dataset elegido en la primera entrega.
2. **Fase de Ingesta y Diagnóstico:** Importa Pandas y carga tus datos. No te limites a ver los datos; utiliza `.shape`, `.info()` y `.describe()` para extraer métricas. En una celda Markdown, redacta un breve análisis (mínimo 2 observaciones) sobre anomalías o tendencias que detectes en la estadística descriptiva (ej. ¿Hay valores en 0 que deberían ser nulos? ¿La desviación estándar es muy alta?).
3. **Cálculo de Datos Faltantes:** Escribe una celda de código que genere un ranking de columnas según su porcentaje de valores nulos, de mayor a menor. Esto te servirá para decidir qué columnas limpiar en el futuro.
4. **Implementación de Lógica Python:**
  - Define una función formal con `def` que incluya un condicional `if/elif/else`. Por ejemplo, crea una función que categorice clientes según su gasto o productos según su precio.
  - Crea una función `lambda` para una transformación atómica (ej. normalizar un texto a minúsculas o redondear una cifra).
5. **Documentación:** Asegúrate de que cada bloque tenga comentarios explicativos sobre por qué estás aplicando esa lógica específica para tu caso de negocio.

**Qué tenés que entregar:** Enlace público a la actividad propuesta, puede ser en un documento de Google o en un repositorio.